

FairJack: On-chain Blackjack with a Staked House Pool

INF0422 – Blockchain, Distributed and Decentralised Systems
[Name Surname; Name Surname]

1. Aim and system model

FairJack is an educational decentralised blackjack prototype. A Solidity contract holds the game’s authoritative state and ERC-20 tokens; a React interface connects players, liquidity providers and application-level validators through a wallet. The test token FJT is freely mintable, so it is used only to demonstrate the protocol, not to represent real value. The project combines an on-chain state machine, shared financial accounting and multi-party commit-reveal randomness.

Liquidity providers deposit FJT into a house pool and receive shares. Their share price changes as the house wins or loses. A player stakes a bet $b \in [1, 100]$ FJT, and the pool reserves $2b$ until the game ends. Thus an ongoing game cannot make its maximum payout unavailable through a withdrawal. Three registered validators, distinct from the player, are assigned to each game; each must have at least 100 FJT of collateral available per active game. Selection starts at an index derived from a hash of the player, game ID and contract address, then scans eligible validators. It is deterministic, not a source of unpredictable randomness.

2. Game protocol

The player and each of the three validators first submit $H(s, \text{gameId}, \text{sender}, \text{contract})$, where s is a private 32-byte secret and H is Keccak-256. Binding the commitment to the game, account and contract prevents reuse in another context. All four commits must arrive within 15 minutes. A further 15-minute reveal phase opens only after the fourth commit. The contract checks every reveal against its commitment and, after all four reveals, hashes the secrets in a fixed order to form the final seed (Fig. 1).

Cards are drawn from a 52-card deck by hashing the seed with a changing nonce; already drawn cards are rejected. Two cards go to the player and two to the dealer. The player may **hit** or **stand**; each valid hit renews a five-minute action deadline. A bust ends the game immediately. After a stand, the contract draws for the dealer until its score reaches at least 17, then settles by comparing scores. A win pays the player $2b$ in total, a push returns b , and a loss transfers the bet’s economic value to the pool.

3. Fairness mechanisms and failure handling

Commit–reveal prevents a participant from changing its secret after seeing other commitments. The pool’s shares and reserved-liquidity check allocate ordinary wins and losses to the stakers while protecting active payouts. Anyone may call `claimTimeout` after a deadline: a validator that misses a commit or reveal loses 100 FJT collateral, credited to the pool; an otherwise compliant player gets the bet back and the game is cancelled. If the player misses its commit, reveal or action, the bet is forfeited. Cancellation and settlement release the game’s liquidity reservation and validator capacity once. The contract uses OpenZeppelin’s `SafeERC20` for token transfers and `ReentrancyGuard` on methods that transfer tokens or settle a game.

4. Evaluation and limitations

The repository includes Hardhat tests for commitment validation, reveal ordering, card uniqueness, scoring, payout accounting, deadlines, slashing and a complete game. These tests specify intended behaviours; this report does not claim an independent security audit or a deployed Sepolia instance. The web interface exposes pool operations, validator management and the game flow (Fig. 2). The source contains a local deployment script and instructions in `README.md`; the same script has a Sepolia configuration.

There are important fairness limits. A participant who reveals last can choose to abort after seeing the other secrets; slashing makes this costly but does not remove the option. Once the seed is published, the remaining deck can be computed, so a player can choose hits with knowledge unavailable in physical blackjack. The dealer’s initial cards are also recorded in public state. Validator selection is predictable and identities are not Sybil-resistant. The FJT token and fixed 18-decimal bet constants further limit the design to a controlled demonstration. These points prevent a claim that the current game is secure for real-money use.

5. Reproduction and AI declaration

The project archive should include the source and `README.md`, which gives the local setup, deployment and browser walkthrough. In brief, install the root and frontend dependencies, start the Hardhat node, run `npm run deploy:local`, then `npm run frontend`; the README lists the required test accounts and wallet network. AI tools supported the testing work throughout the project, including security-oriented cases and checks of code usability, and suggested refactorings where clearer implementations were possible. OpenAI Codex was also used to analyse the repository and draft this report.

References. [1] FairJack source: `contracts/FairJackPool.sol`, `contracts/MockERC20.sol`, `test/`, and `README.md` (submitted version). [2] OpenZeppelin Contracts, ERC-20, SafeERC20 and ReentrancyGuard (dependencies used by the source).

Figures

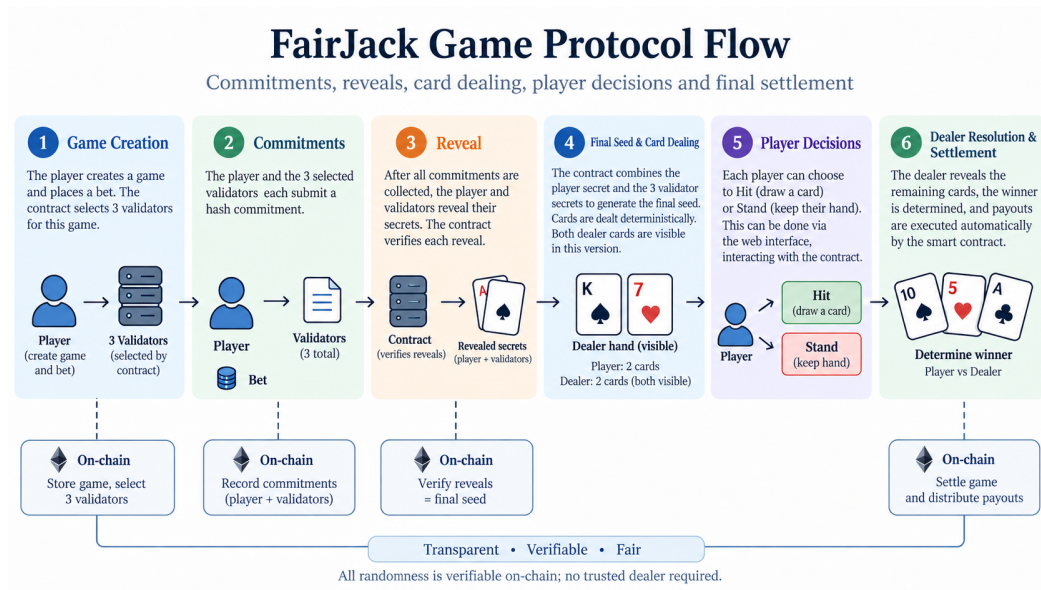


Figure 1. FairJack game lifecycle: commitments, reveals, card dealing, player decisions and final settlement.

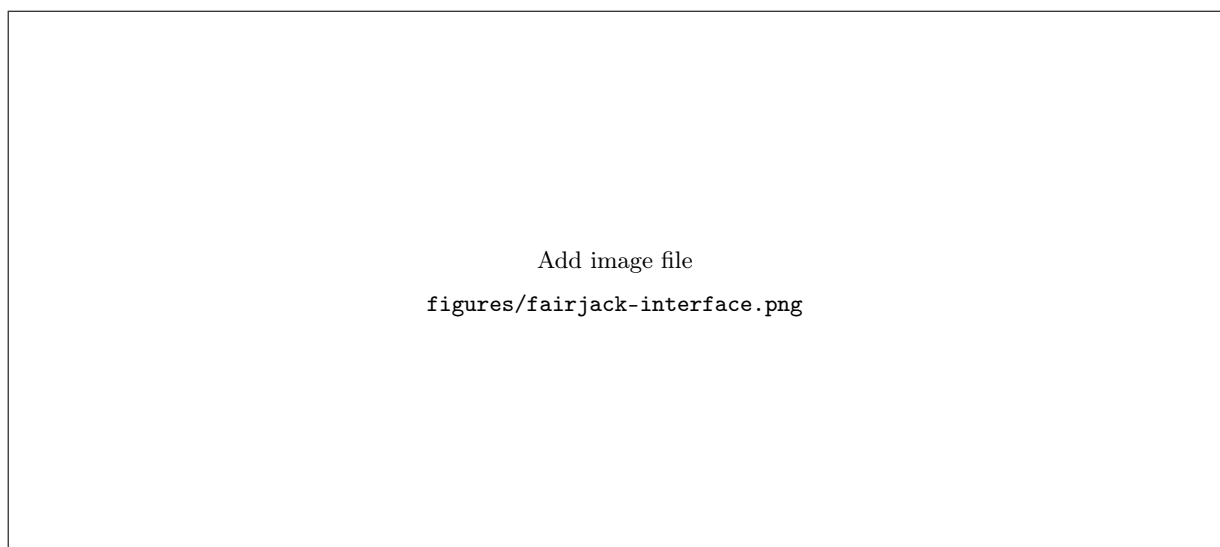


Figure 2. FairJack web interface showing game progress alongside pool or validator status.